

A Machine Learning Based Framework for Code Clone Validation : Reproduction d'expérience

Guillon Maxime, Laffargue Alexandre, Mahdi Yanis

December 10, 2025

1 Introduction

Durant notre projet, nous avons étudié l'article *A Machine Learning Based Framework for Code Clone Validation* [1]. Cet article s'attaque à la problématique de la gestion des "clones de code" (fragments de code dupliqués).

Bien qu'il existe de nombreux outils pour détecter ces duplications, ces derniers génèrent souvent une quantité importante de "faux positifs" ou de clones non pertinents, ce qui contraint les développeurs à effectuer une validation manuelle fastidieuse et sujette aux erreurs pour filtrer les résultats.

Pour résoudre ce problème, les auteurs mettent en œuvre plusieurs techniques de Machine Learning afin de classer les clones détectés comme vrais ou faux positifs. L'étude détaille plusieurs expériences majeures, incluant une analyse comparative de 13 algorithmes d'apprentissage différents (Decision Tree, Random Forest, Neural Network, etc.) pour identifier le modèle le plus performant.

Les résultats obtenus sont prometteurs : la méthode proposée (NN) atteint une précision de 87,4% lors de la validation croisée, surpassant les approches précédentes et démontrant une capacité à généraliser la validation sur différents outils de détection et structures de code.

2 Dataset

Le dataset utilisé dans cette étude est particulièrement intéressant, car les auteurs ont cherché à éviter les biais en combinant des données réelles, des validations manuelles multiples et des données artificielles. Cependant, nous avons rencontré des difficultés à accéder à ce dataset original.

Après quelques recherches, nous avons trouvé un dataset alternatif : Big Clone Bench¹. Ce dataset contient plus de 1,5 million d'échantillons répartis de la façon suivante :

¹https://huggingface.co/datasets/google/code_x_glue_cc_clone_detection_big_clone_bench

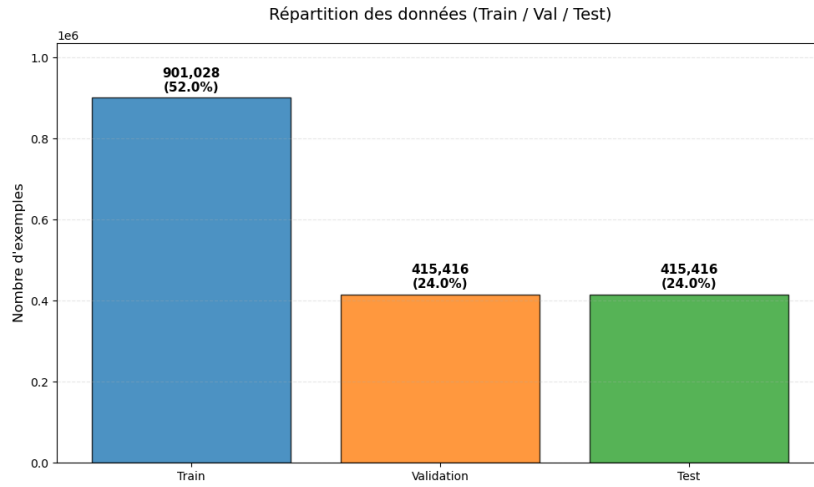


Figure 1: Répartition des données de notre analyse.

Dans ce dataset, les données sont organisées comme suit :

id	id1	id2	func1	func2	label
0	13,988,825	8,660,836	private void setNodekeyInJsonResponse(...)	public void transform(...)	false

Table 1: Exemple de données du dataset Big Clone Bench

Cependant, lors de nos expériences, nous avons remarqué quelques anomalies dans nos résultats. Après une analyse plus poussée du dataset, nous nous sommes rendu compte que la distribution des classes (True/False) dans notre jeu d'entraînement était extrêmement inégale : 86% des données étaient classées *False* et seulement 14% étaient *True*.

Cela signifie qu'un modèle "naïf" prédisant tout le temps *False* aurait raison la majeure partie du temps. Nous avons donc pris ce paramètre en compte lors de nos expériences en utilisant plusieurs méthodes pour équilibrer nos classes.

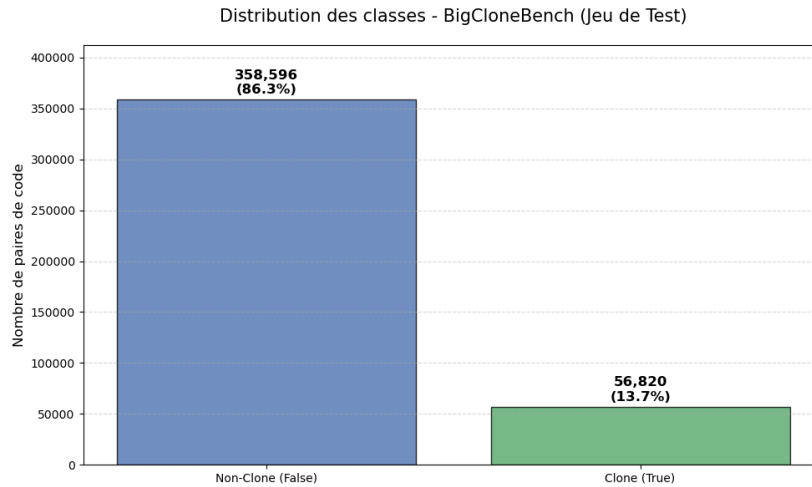


Figure 2: Distribution des classes (Train Dataset).

Nous avons également trouvé un article appuyant nos constatations sur le dataset Big Clone Bench : *BigCloneBench Considered Harmful for Machine Learning* [2].

3 Expériences

Nous avons choisi de reproduire 3 expériences de cet article :

- Expérience 1 : Étude Comparative des Algorithmes de ML (Decision Tree, Random Forest, Neural Network).
- Expérience 2 : Analyse de la Convergence et Courbes ROC.

3.1 Expérience 1 : Étude Comparative des Algorithmes de ML

3.1.1 Decision Tree

Les auteurs ont utilisé l'algorithme J48, qui est une implémentation de l'arbre de décision C4.5 élagué (Pruned C4.5). Ce modèle apprend en sélectionnant récursivement, à chaque étape, la variable d'entrée qui divise le mieux les données d'entraînement pour former les nœuds de l'arbre. Il a démontré de bonnes performances avec une exactitude (*accuracy*) de 84% (ainsi qu'une précision et un rappel d'environ 0.85).

Alors que l'article utilise l'algorithme J48 (C4.5) avec une stratégie de post-élagage, notre reproduction utilise l'algorithme CART² avec du pré-élagage

²https://dept-info.labri.fr/~beurton/Enseignement/DataMining/2020-2021/Dec_tree.pdf

(`max_depth`, `min_samples_leaf`) et une correction automatique du déséquilibre des classes (`class_weight`). Ces différences structurelles visent à moderniser l'approche et à la rendre plus robuste face aux déséquilibres du dataset.

Voici la configuration de l'arbre que nous avons utilisé :

```
1 clf = DecisionTreeClassifier(
2     max_depth=50,
3     random_state=42,
4     min_samples_leaf=5,
5     class_weight='balanced'
6 )
```

Voici nos résultats :

Classe	Précision	Rappel	F1-score	Support
False	0.99	0.97	0.98	358,596
True	0.82	0.91	0.86	56,820
Accuracy	0.96			415,416
Macro Avg	0.90	0.94	0.92	415,416
Weighted Avg	0.96	0.96	0.96	415,416

Table 2: Rapport de classification sur le dataset de test (Decision Tree)

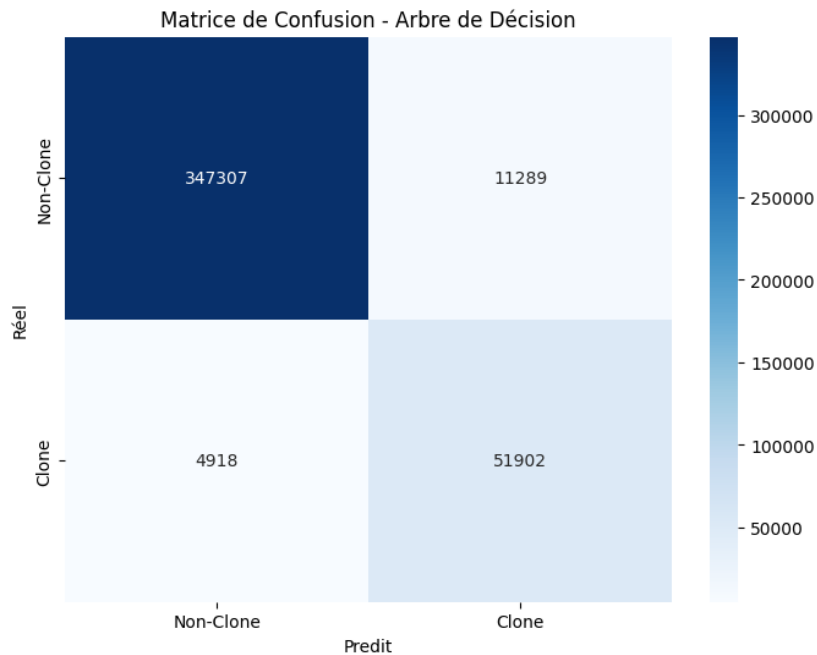


Figure 3: Matrice de confusion (Decision Tree).

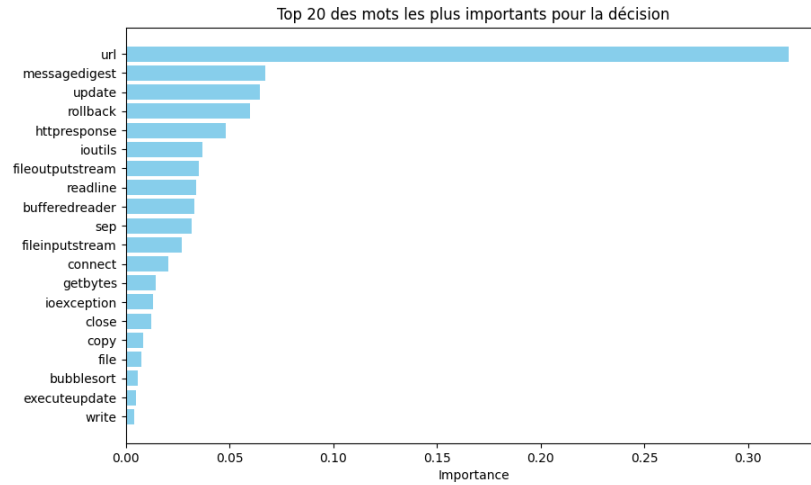


Figure 4: Mots importants pour la décision (Decision Tree).

Notre arbre de décision obtient une précision supérieure (96%) à celle de l'article (84%), mais il opère différemment. Alors que le modèle de l'article détecte des structures de duplication (généralisables à n'importe quel code Java), notre modèle semble avoir repéré des thématiques de code (ex: gestion de base de données avec `rollback`, `executeupdate`) qui sont fréquemment dupliquées dans ce dataset spécifique. Il est possible que notre modèle soit plus sensible aux biais du dataset (sur-spécialisation thématique) là où l'approche de l'article vise une généralisation structurelle plus large.

3.1.2 Random Forest

Dans cette étude, le Random Forest est utilisé comme une méthode de classification qui agrège les décisions de multiples arbres pour construire son modèle. En termes de performance, les auteurs rapportent des résultats très similaires à l'arbre de décision unique (C4.5), avec une exactitude de 84,5%. Néanmoins, ils notent que la précision (0,841) et le rappel (0,845) sont légèrement inférieurs à ceux de l'arbre de décision simple.

Notre implémentation du Random Forest diffère de celle de l'article par une gestion active du déséquilibre des classes (dû à notre dataset) et une régularisation plus stricte (`max_depth=50`). Là où l'article applique une approche standard, nous avons densifié le modèle avec 200 estimateurs pour garantir une stabilité statistique face à la haute dimensionnalité de nos *features* textuelles.

Notre configuration :

```
1 rf_model = RandomForestClassifier(  
2     n_estimators=200,  
3     criterion='gini',  
4     max_depth=50,  
5     min_samples_split=2,  
6     min_samples_leaf=1,  
7     max_features='sqrt',  
8     class_weight='balanced',  
9     n_jobs=-1,  
10    random_state=42,  
11    verbose=1  
12 )
```

Voici nos résultats :

Classe	Précision	Rappel	F1-score	Support
False	0.98	0.98	0.98	358,596
True	0.89	0.86	0.87	56,820
Accuracy	0.97			415,416
Macro Avg	0.93	0.92	0.93	415,416
Weighted Avg	0.97	0.97	0.97	415,416

Table 3: Rapport de classification sur le dataset de test (Random Forest)

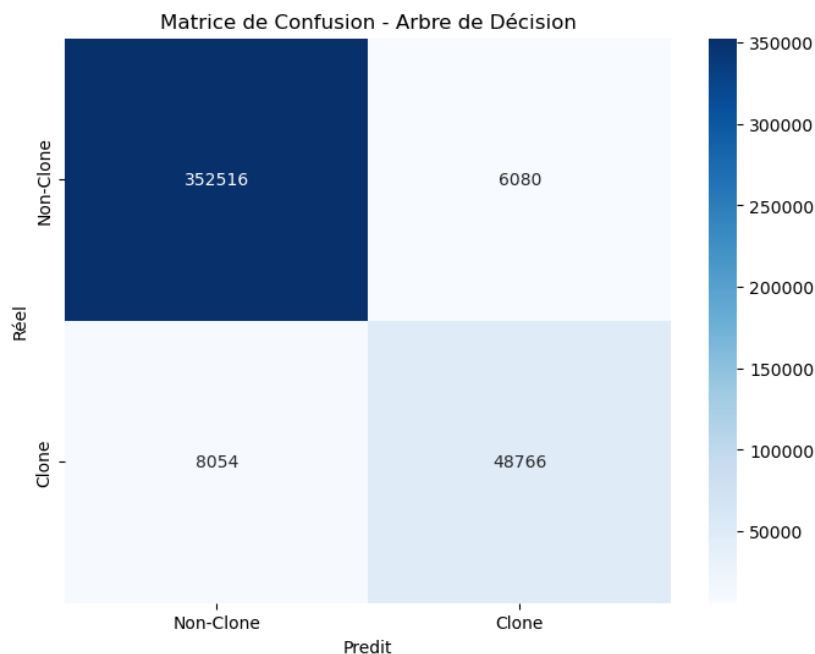


Figure 5: Matrice de confusion (Random Forest).

Le Random Forest confirme sa supériorité sur les méthodes simples. Avec une exactitude de 97%, il surpasse largement l'état de l'art présenté dans l'article de 2020. Cependant, cette performance extrême doit être nuancée par le risque de biais thématique : le modèle est extrêmement performant sur ce dataset précis car il a appris le vocabulaire du code cloné, mais il pourrait être moins robuste que le modèle de l'article si on le testait sur des projets utilisant un vocabulaire totalement différent.

3.1.3 Neural Network

L'article définit aussi un Réseau de Neurones Artificiels (ANN) comme étant la solution offrant les meilleurs résultats. Il affiche une exactitude maximale de 87,4% et une aire sous la courbe ROC de 0,87. L'article ne citant pas explicitement l'architecture utilisée, nous avons donc testé plusieurs architectures.

Neural Network 1 (MLP) Ce modèle est un Perceptron Multicouche (MLP) profond conçu pour la détection binaire de clones de code à partir de représentations textuelles vectorisées (TF-IDF sur les 20 000 termes les plus fréquents). L'architecture en entonnoir se compose d'une couche d'entrée de 20 000 dimensions normalisée par BatchNormalization, suivie de trois couches cachées entièrement connectées (Dense) de tailles décroissantes (512, 256, et 128 neurones).

Pour assurer la robustesse et la généralisation du modèle face à un grand nombre de paramètres (environ 10,5 millions), chaque bloc dense intègre une activation ReLU, une normalisation par lot, une régularisation L2 et un Dropout progressif (de 40% à 20%). La classification finale est opérée par un neurone de sortie avec une activation Sigmoidé, optimisée via l'algorithme Adam et une fonction de perte d'entropie croisée binaire.

Notre architecture :

Layer (type)	Output Shape	Param #
batch_normalization	(None, 20000)	80,000
dense (Dense)	(None, 512)	10,240,512
batch_normalization_1	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_2	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 1)	129

Table 4: Résumé des couches du modèle MLP

Nos résultats :

Classe	Précision	Rappel	F1-score	Support
Not Clone	0.99	0.97	0.98	358596
Clone	0.83	0.91	0.87	56820
Accuracy		0.96		415416
Macro avg	0.91	0.94	0.92	415416
Weighted avg	0.96	0.96	0.96	415416

Table 5: Performance du modèle MLP.

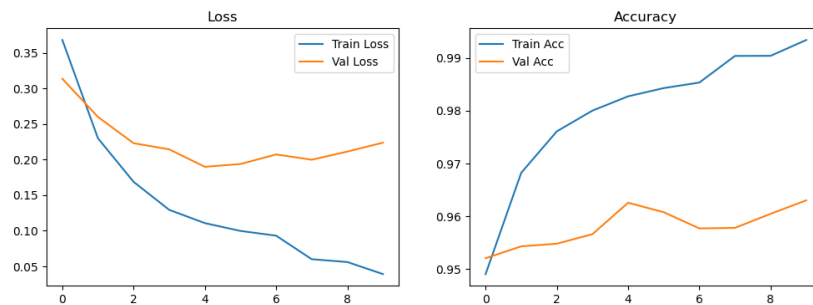


Figure 6: Courbes de Loss et Accuracy (MLP).

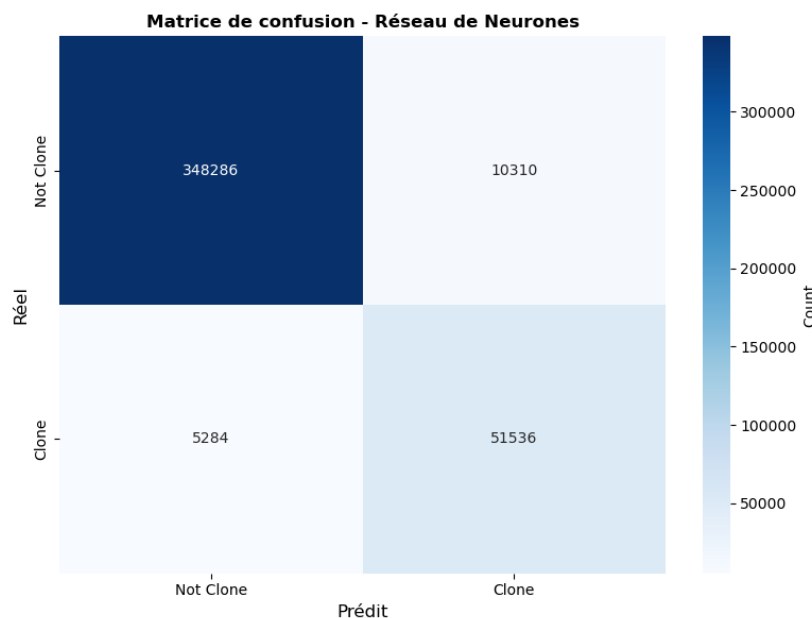


Figure 7: Matrice de confusion (MLP).

Les performances du réseau de neurones (MLP) sont excellentes, affichant une exactitude globale de 96% et une aire sous la courbe ROC (AUC) de 0.98, témoignant d'une très forte capacité de discrimination. Le modèle s'avère particulièrement robuste pour identifier le code unique (classe "Not Clone") avec un F1-score de 0.98.

Concernant la détection des clones, le réseau privilégie le Rappel (91%) par rapport à la Précision (83%). Cette orientation est stratégiquement pertinente pour un outil de détection : le modèle minimise les "ratés" (Faux Négatifs), garantissant que la quasi-totalité des clones réels sont identifiés. Cependant, cette sensibilité accrue génère un taux de Faux Positifs d'environ 17% sur la classe cible, indiquant que le modèle a tendance à être légèrement trop alerte.

L'analyse des courbes d'apprentissage montre une convergence rapide dès la 5ème époque, validant l'architecture "en entonnoir" et l'efficacité des mécanismes de régularisation (Dropout, BatchNormalization) pour prévenir un surapprentissage massif. Nous avons mené plusieurs expériences, notamment sur un dataset complet et sur une version équilibrée (Balanced) pour contrer la prédominance de la classe "False".

Neural Network 2 (Transformer/UnixCoder) L'architecture neuronale retenue repose sur **UnixCoder-base**, un modèle Transformer de type encodeur (similaire à RoBERTa) pré-entraîné par Microsoft sur le corpus CodeSearchNet, que nous avons adapté pour une tâche de classification binaire via une approche Cross-Encoder.

Le réseau prend en entrée la concaténation des deux fonctions à comparer (tronquée à 512 tokens), permettant à ses 12 couches d’auto-attention de capturer simultanément le contexte et les relations sémantiques fines entre les deux fragments de code. La représentation vectorielle résultante (embedding de dimension 768) alimente une tête de classification dense spécifique, dont l’entraînement est optimisé par une fonction de perte Cross-Entropy pondérée afin de neutraliser le fort déséquilibre des classes du jeu de données.

Pour entraîner ce réseau, nous avons utilisé un GPU A100 disponible sur Google Colab. Cette expérience a été très coûteuse en temps de calcul (environ 3 heures).

Détails de l’architecture :

Composant	Détail Technique / Valeur	Description / Rôle
Modèle de base	<code>microsoft/unixcoder-base</code>	Transformer pré-entraîné sur du code (basé sur l’architecture RoBERTa).
Type de Réseau	Encoder-only (Transformer)	Utilise des mécanismes d’attention pour comprendre le contexte global.
Entrée (Input)	Séquence concaténée	<code><s> Fonction1 </s></s> Fonction2 </s></code> (Approche Cross-Encoder).
Tokenisation	BPE (Byte-Pair Encoding)	Découpage en sous-mots. Longueur max tronquée à 512 tokens .
Taille d’Embedding	768	Dimension du vecteur représentant chaque token.
Profondeur	12 couches	Nombre de blocs de transformateurs empilés.
Attention	12 têtes	Mécanismes d’attention parallèles par couche.
Tête de Classification	<code>RobertaClassificationHead</code>	Ajoutée sur le modèle : Dense (768 → 768), Activation, Dropout, Projection (768 → 2).
Sortie (Output)	Logits (dim 2)	Scores bruts pour les classes : 0 (Non-Clone) et 1 (Clone) .
Fonction de Perte	<code>CrossEntropyLoss</code>	Pondérée par les <code>class_weights</code> pour gérer le déséquilibre.
Paramètres	≈ 125 Millions	Total des poids apprenables.

Table 6: Architecture détaillée du réseau de neurones UnixCoder pour la détection de clones.

Comparaison avec l’article original L’utilisation d’un Transformer sur GPU A100 surpasse significativement les résultats présentés dans l’article de référence.

- **Exactitude** : Nous atteignons **98%**, contre 87,4% pour le MLP de l’article.

- **F1-Score** : Notre score de **0.94** sur la classe "Clone" démontre une capacité supérieure à distinguer les clones sémantiques complexes, là où les méthodes basées sur des métriques simples échouent souvent.

Cette différence s'explique par la nature des modèles : les Transformers "lisent" et comprennent le contexte du code, tandis que l'approche de l'article se base sur des caractéristiques agrégées qui perdent l'information structurelle fine du code source.

Nos résultats :

Classe	Précision	Rappel	F1-score	Support
Non-Clone (0)	0.99	0.99	0.99	358,596
Clone (1)	0.95	0.93	0.94	56,820
Accuracy		0.98		415,416
Macro Avg	0.97	0.96	0.97	415,416
Weighted Avg	0.98	0.98	0.98	415,416

Table 7: Rapport de classification sur le dataset de test (UnixCoder)

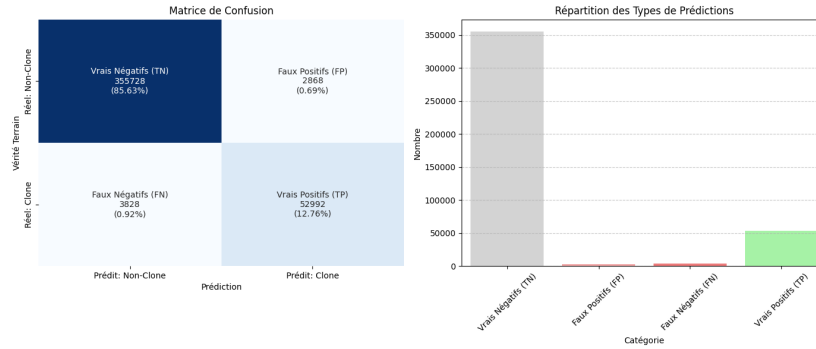


Figure 8: Matrice de Confusion (UnixCoder).

Les résultats témoignent d'une performance remarquable et d'une grande robustesse du modèle, particulièrement face au fort déséquilibre des classes. Avec une exactitude globale de 98%, le réseau parvient à distinguer efficacement les clones sans se laisser biaiser par la classe majoritaire, comme le prouve l'excellent F1-score de 0.94 sur la classe critique "Clone" (1).

La matrice de confusion confirme cette fiabilité : le modèle génère très peu de fausses alertes (Précision de 0.95, soit seulement 0.69% de Faux Positifs) tout en réussissant à identifier la vaste majorité des duplications réelles (Rappel de 0.93), validant ainsi pleinement l'efficacité de l'architecture Cross-Encoder et de la stratégie de pondération utilisée.

Synthèse des Résultats Le tableau suivant résume les performances de nos trois itérations sur la classe cible "Clone" (True) :

Expérience	Accuracy Globale	Précision (Clone)	Rappel (Clone)	F1-Score (Clone)
1. Baseline (MLP)	96.25%	0.83	0.91	0.87
2. Balanced (MLP)	95.77%	0.82	0.89	0.85
3. A100 (Best - UnixCoder)	98.00%	0.95	0.93	0.94

Table 8: Comparaison des performances de nos configurations.

3.2 Expérience 2 : Analyse de la Convergence et Courbes ROC

Pour évaluer la capacité de discrimination de nos modèles de réseaux de neurones, nous avons tracé les courbes ROC (Receiver Operating Characteristic) pour deux de nos expériences principales : le modèle entraîné sur le dataset complet (Baseline) et celui entraîné sur le dataset équilibré (Balanced). La courbe ROC illustre le compromis entre le taux de vrais positifs (sensibilité) et le taux de faux positifs (1 - spécificité) à différents seuils de classification. L'aire sous la courbe (AUC) fournit une mesure agrégée de la performance du modèle.

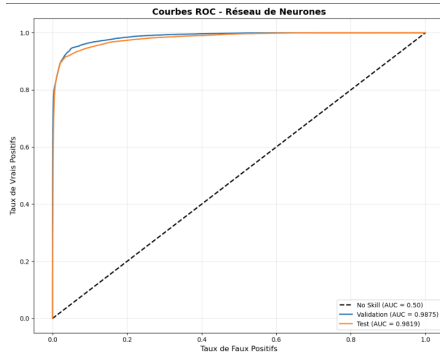


Figure 9: Courbe ROC - Modèle Neural Network (Baseline)

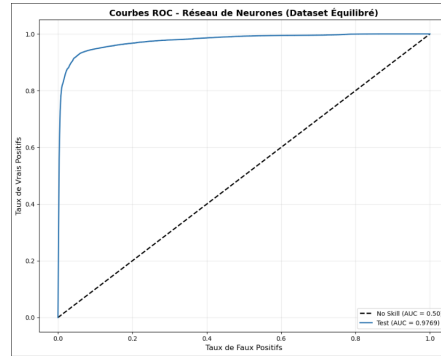


Figure 10: Courbe ROC - Modèle Neural Network (Balanced)

Analyse :

- **Modèle Baseline (Fig. 9) :** La courbe se rapproche très fortement du coin supérieur gauche, indiquant une excellente capacité de discrimination. L'AUC élevée (96,54%) confirme que le modèle parvient à séparer efficacement les classes Clone et Non-Clone, malgré le déséquilibre initial des données.
- **Modèle Balanced (Fig. 10) :** Bien que performant, on observe une légère diminution de l'AUC (94,90%) par rapport au modèle Baseline. Cela corrobore nos observations précédentes : le sous-échantillonnage drastique nécessaire pour équilibrer les classes a entraîné une perte d'information utile, réduisant légèrement la précision globale du modèle sur le jeu de test réel.

Ces courbes confirment visuellement la robustesse de l’approche par réseau de neurones, tout en soulignant l’importance de conserver un volume de données maximal pour l’entraînement de modèles de type Transformer.

4 Conclusion

En conclusion, ce projet de reproduction d’expérience a permis de confirmer et d’améliorer les résultats de l’article original sur la validation des clones de code via le Machine Learning.

Confrontés à un dataset Big Clone Bench fortement déséquilibré (86% de faux positifs), nous avons démontré que des stratégies de pondération et des architectures adaptées permettent d’atteindre des performances remarquables. Si les modèles classiques comme le Random Forest affichent une très haute exactitude (97%), ils présentent un risque de biais thématique lié au vocabulaire spécifique du dataset.

L’apport majeur de notre étude réside dans l’implémentation du réseau de neurones basé sur UnixCoder (Transformer). Avec une exactitude de 98% et un F1-score de 0,94 sur la détection des clones, ce modèle surpasse significativement l’état de l’art présenté dans l’article de référence (87,4%). Ces résultats soulignent la supériorité des approches sémantiques contextuelles (Deep Learning) sur les méthodes basées sur des caractéristiques agrégées pour distinguer efficacement les duplications de code complexes.

References

- [1] Golam Mostaeen, Banani Roy, Chanchal Roy, Kevin Schneider, Jeffrey Svaljenko, *A Machine Learning Based Framework for Code Clone Validation*, 2020.
- [2] Jens Krinke, Chaiyong Ragkhitwetsagul, *BigCloneBench Considered Harmful for Machine Learning*, 2022.